CloudSentinel AI: A Context-Aware Framework for Cloud Misconfiguration Risk Assessment and Candidate Attack-Path Analysis

Harshith

***Abstract*—Cloud Security Posture Management (CSPM) tools commonly report findings at the individual-resource level, which can obscure relationships among cloud assets and complicate remediation prioritisation. This paper proposes CloudSentinel AI, an AWS-focused framework for context-aware cloud misconfiguration assessment. The design collects selected configuration metadata, models assets and relationships in a knowledge graph, evaluates documented security rules, and derives candidate attack paths subject to explicit preconditions. A transparent contextual scoring heuristic is proposed to prioritise findings using asset criticality, exposure, identity permissions, data sensitivity, and graph context. An LLM-assisted reporting component is constrained to evidence-bounded, human-reviewed explanations and remediation recommendations. The paper also defines a controlled AWS evaluation protocol based on labelled scenarios. A tagged, isolated AWS baseline has been deployed, while a private S5 workload was temporarily deployed, configuration-verified, assessed by read-only evidence-collection passes, and then torn down. The passes detected a bounded security-group configuration finding and a temporary bucket-scoped IAM wildcard permission, while rejecting candidate paths whose reachability preconditions were absent. A private S5 instance-profile relationship to a synthetic S3 object and metadata-only secret was observed, but no candidate path was asserted. An account-level S3 safeguard blocked attachment of the planned empty-bucket public policy. A dedicated S4 CloudTrail test trail was also observed in the disabled state without altering an existing trail. A deterministic local S6 precondition model also rejected the hypothetical SSRF-to-IAM path without contacting AWS or a network; no S6 service was deployed. Comparative scenario-level experiments remain unexecuted. This work therefore does not claim completed experimental validation or general production effectiveness.**

***Index Terms*—cloud security, cloud misconfiguration, AWS, knowledge graph, attack-path analysis, risk prioritisation, LLM-assisted security reporting**

I. Introduction

Cloud computing provides on-demand access to configurable computing resources over standard network protocols [1], transforming enterprise infrastructure through rapid resource provisioning and multi-tenant resource pooling. However, the shared responsibility model and dynamic orchestration intrinsic to modern cloud architectures introduce substantial security challenges [2], [3]. Security posture in cloud environments does not depend solely on individual asset configurations; rather, it emerges from the intricate interactions among identity controls, network routing boundaries, object storage access policies, audit logging, and interdependent service relationships [4], [5].

Unlike conventional software vulnerabilities resulting from source-code programming errors, cloud security incidents predominantly stem from security misconfigurations, overly permissive identity privileges, and defective Infrastructure-as-Code (IaC) deployment definitions [6], [7]. Contemporary Cloud Security Posture Management (CSPM) tools typically assess cloud accounts by scanning isolated assets against static compliance rules. While effective at cataloguing individual compliance violations, this resource-centric paradigm produces thousands of atomised alerts that obscure underlying architectural relationships [8]. An isolated configuration finding does not convey its operational significance without contextual grounding: for example, the practical risk posed by an open security-group ingress rule depends fundamentally on whether the associated workload is publicly routable, what IAM role is attached, which sensitive data stores are reachable, and what defensive controls remain active along potential traversal paths.

CloudSentinel AI is proposed as an AWS-focused research framework that combines multi-service configuration assessment with relationship-aware reasoning. The study follows a Design Science Research (DSR) methodology in which the framework serves as the purposeful artifact and the controlled evaluation protocol empirically tests its bounded claims [9]. The primary contributions of this paper are:

1) A typed AWS resource-and-relationship model for building a security knowledge graph from configuration metadata across EC2, IAM, S3, VPC, RDS, Secrets Manager, and CloudTrail.
2) A candidate attack-path analysis algorithm that enumerates bounded, evidence-supported traversal sequences while strictly validating explicit reachability and privilege preconditions.
3) A transparent contextual risk prioritisation heuristic that scores findings using documented severity, asset criticality, attack-path context, and network exposure without black-box opacity.
4) An evidence-bounded LLM-assisted reporting contract that generates human-reviewed explanations and advisory remediation guidance grounded exclusively in verified configuration facts.

The framework is a design proposal. Any empirical claim of superior finding detection, alert deduplication, or analyst efficiency requires the controlled evaluation design formalized

in this paper.

II. Background and Related Work

A. Cloud Misconfiguration Detection and IaC Verification

Configuration errors represent a leading operational risk across public cloud infrastructures [6]. A substantial body of literature addresses static configuration verification in Infrastructure-as-Code (IaC) templates prior to deployment. Rahman et al. [7] categorized recurring security smells in IaC scripts, demonstrating that hardcoded secrets and unconstrained administrative privileges frequently escape code review. Chiari et al. [10] provided a comprehensive survey of static analysis techniques for IaC, highlighting the trade-offs between pre-deployment policy validation and post-deployment runtime verification. In containerized cloud deployments, Bose et al. [11] and Rahman et al. [12] conducted empirical investigations of security misconfigurations in Kubernetes manifests, documenting widespread authorization flaws and insecure network policies.

In the storage domain, Amazon S3 bucket exposures remain a frequent source of enterprise data breaches. Olorunlana [13] introduced a policy-enforced framework for preventing Amazon S3 cloud storage misconfigurations using Infrastructure-as-Code validation, emphasizing automated policy enforcement to block unintentional public exposures. However, pre-deployment static analysis alone cannot detect configuration drift, dynamic privilege assumptions, or multi-resource interactions arising in active cloud tenancies. To address operational drift, Torkura et al. [4] proposed continuous auditing and threat detection architectures across multi-cloud environments. Similarly, van Ede et al. [8] developed anomaly detection techniques to identify anomalous misconfigurations within AWS Identity and Access Management (IAM) policies, underscoring the necessity of policy reasoning over complex identity structures.

B. Graph-Based Modeling and Attack-Path Analysis

Knowledge graphs provide a flexible, formal representation for capturing entities, attributes, and multi-relational dependencies [14]. In cybersecurity, graph representations have long been leveraged to model networked systems and potential exploitation sequences. Foundational work by Phillips and Swiler [15] introduced graph-based network vulnerability analysis, establishing the conceptual basis for modern attack graphs. Sheyner et al. [16] formulated automated generation and analysis of attack graphs using model-checking techniques, while Jha et al. [17] provided formal analyses to determine minimal cut-sets and vulnerability minimization strategies.

In cloud computing environments, attack-graph methods have evolved to accommodate dynamic virtualization and cloud-native identities. Sabur et al. [18] proposed scalable graph-based security analysis frameworks tailored for cloud networks, addressing combinatorial state-space explosion through modular graph partitioning. Elmiger et al. [19] demonstrated the practical utility of graph reasoning for discovering critical attack paths within Microsoft cloud tenant environments, revealing multi-hop traversal risks that isolated security checks failed to surface. Extending graph reasoning to audit telemetry, Kurniawan et al. [20] proposed KRYSTAL, a knowledge graph-based framework for tactical attack discovery in audit logs. While these approaches motivate graph-based security reasoning, they frequently treat topological reachability as equivalent to exploitability. CloudSentinel AI distinguishes structural graph connectivity from verified attack paths by demanding that every edge transition satisfy explicit, recorded preconditions.

C. Machine Learning, Explainable AI, and Large Language Models

Machine learning techniques have been widely explored for cloud threat detection, anomaly classification, and security log analysis, as surveyed by Nassif et al. [21]. However, complex machine learning classifiers frequently operate as black boxes, providing predictions without interpretable explanations. To overcome this limitation, Explainable Artificial Intelligence (XAI) has emerged as an essential discipline in cybersecurity [22]. Charmet et al. [23] examined XAI literature in cybersecurity, emphasizing that security analysts require transparent rationales to justify incident triage and remediation decisions. Foundational model-agnostic explanation techniques such as LIME [24] and SHAP [25] demonstrate the importance of local feature attribution and transparent scoring components.

Concurrently, generative Large Language Models (LLMs) founded on the Transformer architecture [26] have demonstrated strong few-shot reasoning capabilities across diverse tasks [27], with advanced systems such as GPT-4 [28] demonstrating proficiency in technical summarization and code generation. Xu et al. [29] provided a systematic literature review of LLMs in cybersecurity, detailing applications in vulnerability explanation, log triage, and remediation advice. Nevertheless, LLMs exhibit well-documented propensities for hallucination, unsupported factual assertion, and hazardous remediation recommendations when unconstrained [29].

D. Research Gaps and Proposed Approach

Existing literature reveals three principal research gaps: (i) CSPM tools and static IaC scanners evaluate isolated resources without modeling inter-resource relationship chains; (ii) traditional attack-graph approaches often presume reachability without validating cloud-specific runtime preconditions; and (iii) emerging AI and LLM security tools lack deterministic evidence bounding, risking hallucinations in mission-critical remediation. CloudSentinel AI directly bridges these gaps by synthesizing multi-service AWS configuration analysis, precondition-guarded candidate-path discovery, transparent contextual prioritisation, and an evidence-bounded LLM reporting contract within a reproducible evaluation methodology.

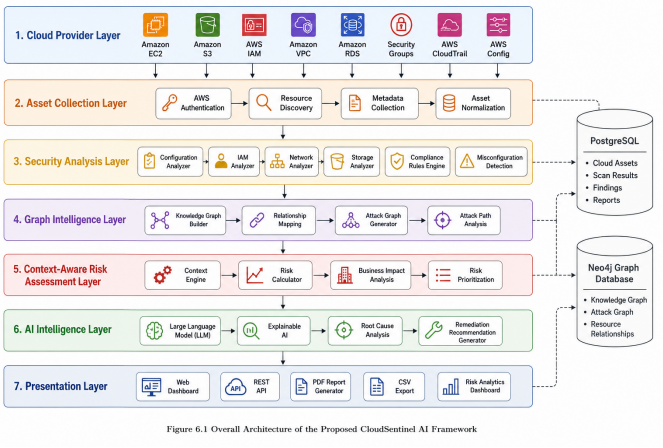


Figure 6.1 Overall Architecture of the Proposed CloudSentinel AI Framework

Fig. 1. High-level CloudSentinel AI architecture. The figure illustrates the proposed pipeline; it is not an implementation-performance result.

III. Proposed Framework

A. Scope and Threat Model

The initial scope of CloudSentinel AI encompasses an AWS account containing EC2, S3, IAM, VPC/security-group, RDS, Secrets Manager, and CloudTrail configuration metadata. The framework operates under read-only collection credentials, evaluating solely the resources visible under authorized API queries. The threat model begins with an external or authenticated actor situated at a documented entry condition (e.g., a public network interface or compromised test principal) and admits transitions only when fully supported by collected configuration evidence. It does not assert an initial compromise, credential theft, exploit availability, or successful lateral movement unless those preconditions are explicitly represented, verified against metadata, and independently reviewed.

B. Architecture and Graph Schema

The proposed processing pipeline consists of five sequential stages. First, an asset collector retrieves versioned AWS configuration metadata using read-only APIs. Second, a configuration analyzer evaluates versioned deterministic rules derived from established industry benchmarks. Third, a graph builder constructs typed nodes and directed edges representing cloud assets, configuration findings, and architectural dependencies. Fourth, a candidate-path engine searches the graph under the specified threat model. Finally, an LLM-assisted reporting component compiles findings, path evidence, scoring rationale, and advisory remediation guidance. Table I defines the minimum graph schema, and Fig. 1 illustrates the pipeline.

C. Candidate-Path Analysis and Prioritisation

A graph edge represents a documented relationship or security-relevant condition; it does not independently prove exploitability [16], [18]. A candidate path is a bounded, loop-free sequence from a documented entry condition to a target asset [19]. The engine will enumerate at most k paths with a maximum depth d, reject paths with a missing required precondition, and retain evidence for every edge. It therefore performs bounded candidate-path discovery rather than claiming exhaustive enumeration of all attack paths.

TABLE I
Minimum Proposed Graph Schema

Element	Required attributes or semantics
Nodes	AWS resource, identity, policy, security finding, network boundary, or data asset; each has an account, region, source timestamp, and evidence identifier [14].
Edges	Typed relationship such as *attached-to*, *permits*, *can-reach*, *can-assume*, *stores*, or *has-finding*; every edge identifies its source metadata [20].
Findings	Rule identifier, affected node, observed evidence, severity mapping, and collection time.
Path record	Ordered edges, entry condition, required preconditions, target, supporting findings, and review status.

To provide transparent risk scoring aligned with interpretability principles [24], [25], the proposed prioritisation score is formulated as a linear combination of normalized factors:

$$R = \alpha S + \beta C + \gamma P + \delta E, \tag{1}$$

where all components are normalized to $[0, 1]$: S is documented finding severity, C is asset criticality, P is evidence-supported candidate-path context, and E is externally reachable exposure. The non-negative weights satisfy $\alpha+\beta+\gamma+\delta = 1$ and will be fixed before evaluation. Exposure is excluded from C to prevent double counting. R is a deterministic prioritisation score, not an estimated probability of compromise; sensitivity analysis will report how ranking shifts under alternative weighting schemes.

D. LLM-Assisted Reporting Contract

The reporting component receives only approved, evidence-bounded findings, graph relationships, and remediation context. Each generated report must distinguish observed evidence from assumptions, identify missing preconditions, link each recommendation to relevant findings, and state that human review is required. Prompt engineering cannot guarantee factual accuracy or eliminate hallucinations [28], [29]; the component is advisory and will be evaluated for evidence consistency, technical accuracy, clarity, and actionability. It must not automatically execute remediation or receive unapproved sensitive data. Table II summarizes the claim-to-evidence evaluation mapping.

IV. Controlled Evaluation Design

The proposed evaluation uses an isolated, authorized, non-production AWS account. All resources will be tagged, synthetic or non-sensitive data will be used, least-privilege roles and budget alarms will be configured, and every scenario will have a verified cleanup record. Intentionally weak configurations will remain within the defined laboratory boundary and will not be exposed to the public Internet, adhering to established design-science evaluation principles [9].

TABLE II
CLAIM-TO-EVIDENCE MAP FOR EVALUATION

Claim	Required evaluation evidence and limitation
Finding detection	Scenario ground truth, normalised scan output, and finding-level precision/recall; limited to selected rules and scenarios.
Candidate-path quality	Expected-path catalogue, edge evidence, and independent path review; graph connectivity alone is insufficient.
Prioritisation	Predefined reference ranking and sensitivity analysis; R is a heuristic, not compromise probability.
LLM-assisted reporting	Human rubric for evidence consistency, accuracy, clarity, and actionability; output remains advisory.

A. Ground Truth and Units of Analysis

The evaluation distinguishes three units. A *finding* is one rule violation attached to a resource. A *candidate path* is an ordered graph sequence with documented preconditions. A *consolidated alert* groups related findings for a shared path or asset context. Before each run, a scenario catalogue (Table III) records the intentional configuration state, expected findings, negative controls, expected and forbidden candidate paths, severity rationale, and permitted remediation guidance. This catalogue forms the ground truth; it must be frozen before scanning and independently reviewed [6], [12].

B. Metrics and Path Review

Finding-level precision, recall, F1, and false-positive rate are computed only against the labelled scenario finding set:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP+FP}, \\ \text{Recall} &= \frac{TP}{TP+FN}, \\ F1 &= \frac{2TP}{2TP+FP+FN}. \end{aligned} \tag{2}$$

The false-positive rate is $FP/(FP+TN)$ when negative controls define TN. A candidate path is an *exact match* when its entry condition, ordered evidence-backed edges, target, and required preconditions match the catalogue; it is a *partial match* when it represents a valid subset or superset with the difference stated; otherwise it is *invalid*. Prioritisation quality will be measured against a pre-specified reference ranking, while alert reduction will be reported as consolidated alerts divided by normalised baseline findings for the same scenario. These measures must not be aggregated across units of analysis.

C. Execution and Baseline Protocol

For every scenario, the study will archive the pre-scan configuration snapshot, resource inventory, ground-truth catalogue, tool version, enabled checks, account scope, permissions, execution time, raw output, graph export, candidate-path evidence, and cleanup record. CloudSentinel and selected baselines will use the same account scope, permissions, configuration, and execution date. Findings will be normalised to the defined finding unit before comparison. Each timing measurement will use at least three non-warm-up repetitions and report the timing boundary, median, range, failures, host specification, AWS region, and any external API latency. A local in-memory fixture runner validates catalogue handling and attack-graph integration before cloud deployment; its outputs are regression checks and are excluded from all cloud-performance and accuracy metrics. Capability matrices based on vendor documentation will be labelled as documentation-based feature comparisons, not benchmark performance results.

D. LLM-Report Review

Each LLM-assisted report will be evaluated by at least two independent reviewers using a pre-defined rubric. The rubric scores: (i) evidence consistency, (ii) technical accuracy, (iii) clarity, and (iv) actionability. Reviewers will mark unsupported statements and unsafe recommendations separately, report agreement before adjudication, and retain the original model output and prompt version. If external participants are recruited, the study will obtain the required institutional ethics approval before data collection.

E. Research Questions and Reproducibility Artifacts

The study evaluates four bounded research questions: ***RQ1***, whether the selected rule set detects the labelled findings in S1–S6; ***RQ2***, whether generated candidate paths satisfy the expected-path conditions; ***RQ3***, whether the heuristic ranking agrees with the pre-specified reference ranking; and ***RQ4***, whether LLM-assisted reports meet the reviewer rubric without unsupported or unsafe recommendations. The versioned scenario catalogue, ground-truth definitions, and validation script are maintained in the repository root under `evaluation-scenarios/`. For each executed run, the study will archive sanitised configuration snapshots, source and tool versions, normalised outputs, graph evidence, measurement records, and cleanup confirmation. Secrets, credentials, unredacted logs, and sensitive asset identifiers are excluded from the repository.

V. IMPLEMENTATION STATUS AND BASELINE DEPLOYMENT EVIDENCE

At the time of writing, the controlled AWS baseline remains deployed in an isolated evaluation account in the `ap-south-2` region and is managed by Terraform. A private S5 instance, metadata-only secret, synthetic S3 object, subnet, security group, and instance profile were temporarily deployed for measurement and then removed by Terraform. The resource identifiers, execution artifacts, and account-specific logs are retained outside the repository; only sanitised evidence is archived.

Read-only AWS API checks verified the state in Table IV, and a subsequent Terraform plan reported no pending resource changes. The baseline pass produced one medium-severity

TABLE III
CONTROLLED AWS SCENARIO CATALOGUE

ID	Scenario	Evaluation focus
S1	Public S3 storage exposure	Finding detection, evidence capture, and remediation guidance for a test-only bucket [13].
S2	Overly permissive IAM policy	Detection of test-principal permissions and supported identity relationships [8].
S3	Insecure network configuration	Detection of a laboratory security-group condition and documented reachability preconditions [18].
S4	Disabled CloudTrail logging	Detection of an audit-logging condition and restoration recommendation [4].
S5	Multi-stage candidate path	Evidence-backed relationship reasoning across test EC2, IAM, Secrets Manager, and S3 resources [19].
S6	SSRF-to-IAM credential-access candidate path	Planned local precondition-only analysis of configuration and identity conditions; no deployed service, public exposure, production credentials, or metadata requests.

TABLE IV
VERIFIED CONTROLLED-BASELINE STATE

Component	Observed configuration verification
S3 controls	All four S3 public-access blocks enabled; AES-256 default encryption enabled; the temporary S5 synthetic object was removed after measurement.
IAM role	Dedicated role has no attached managed policies; after S5 teardown its inline policy is limited to bucket listing/object read.
Network boundary	VPC has no Internet gateway or external route; the temporary S5 private subnet was removed after measurement.
S5 measurement	One private `t3.micro` instance was assessed without a public address; IMDSv2 required with hop limit one; encrypted root volume; all S5-only resources were then removed.
Security-group fixture	The S3 fixture has one `tcp/8443` ingress rule from `0.0.0.0/0`; the temporary S5 group had no ingress or egress.

configuration finding for the security-group ingress rule that permits `0.0.0.0/0` as a source. It produced no candidate path: the required preconditions—an Internet gateway, subnet, and network interface attached to the group—were all absent. A separate temporary S1/S2 configuration pass disabled the bucket-level S3 public-access blocks and changed the test role to `s3:*` on the empty test bucket. The evaluator detected both high-severity configuration conditions and still produced no candidate path. The intended S1 public `GetObject` policy was rejected by an account-level S3 Block Public Access safeguard; no object, public policy, or public data was created. Terraform then restored all four bucket public-access blocks and the role's `ListBucket`/`GetObject`-only policy. For S4, a separate dedicated test trail was created in the disabled state; its `IsLogging` status was false, while its private log bucket retained all public-access blocks and AES-256 default encryption. No existing trail was stopped or modified, and the dedicated trail was never enabled. The S5 pass observed the private instance-profile relationship to the synthetic object and metadata-only secret, while confirming no public address, no ingress or egress, no Internet or NAT route, IMDSv2 with hop limit one, and no `GetSecretValue` permission. It therefore asserted no candidate path, after which Terraform removed all S5-only resources and restored the role policy. These passes demonstrate bounded evidence collection and precondition rejection, not aggregate S1–S6 detection accuracy, candidate-path quality, timing, baseline-tool comparison, or LLM-report quality. The deterministic local S6 model likewise rejected the hypothetical path with no AWS or network access; it is not a deployed-service experiment.

VI. DISCUSSION AND LIMITATIONS

The proposed framework does not establish that a graph path is exploitable merely because related resources are connected. The threat model, evidence sources, path-validity rules, and score weights must be explicit. The controlled scenarios also do not represent all production environments, cloud providers, threat actors, or organisational policies [6], [7]. The initial scope is AWS only, and the in-memory graph design may not scale to ultra-large deployments without distributed graph partitioning [18].

Furthermore, while continuous auditing architectures [4] and multi-cloud federations [5] are increasingly common in enterprise practice, expanding cross-provider semantics remains future work. The LLM component adds cost, latency, privacy, and reliability concerns. It must not receive unapproved sensitive data, and its recommendations must not be executed automatically. These limitations motivate a staged evaluation in which each contribution is tied to an implementation artifact, metric, evidence source, and stated limitation.

VII. CONCLUSION

This paper presented CloudSentinel AI, a proposed framework for combining AWS configuration assessment, knowledge-graph modeling, candidate attack-path analysis, contextual prioritisation, and LLM-assisted reporting. The central contribution is a reproducible evaluation design that separates labelled finding detection, candidate-path validity, prioritisation, and explanation quality. Future work will integrate the read-only evidence collector into the product pipeline, complete the remaining rule implementations and any separately approved S6 private instrumentation, archive sanitised evidence, and report bounded results without generalising beyond the evaluated configurations.

REFERENCES

[1] P. Mell and T. Grance, "The nist definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. SP 800-145, 2011.

[2] B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, and M. Ayaz, "A systematic literature review on cloud computing security: Threats and mitigation strategies," *IEEE Access*, vol. 9, pp. 57 792–57 807, 2021.

[3] R. Kumar and R. Goyal, "On cloud security requirements, threats, vulnerabilities and countermeasures: A survey," *Computer Science Review*, vol. 33, pp. 1–48, 2019.

[4] K. A. Torkura, M. I. Sukmana, F. Cheng, and C. Meinel, "Continuous auditing and threat detection in multi-cloud infrastructure," *Computers & Security*, vol. 102, p. 102124, 2021.

[5] D. Saxena, R. Gupta, and A. K. Singh, "A survey and comparative study on multi-cloud architectures: Emerging issues and challenges for cloud federation," *arXiv preprint arXiv:2108.12831*, 2021.

[6] Y. Liu, Y. Zhou, H. Zhang, Z. Chang, S. Xu, Y. Jia, W. Wang, and Z. Liu, "Rethinking software misconfigurations in the real world: An empirical study and literature analysis," 2024.

[7] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.

[8] T. van Ede, N. Khasuntsev, B. Steen, and A. Continella, "Detecting anomalous misconfigurations in aws identity and access management policies," in *Proceedings of the 2022 ACM Cloud Computing Security Workshop*, 2022, pp. 63–74.

[9] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[10] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: A survey," in *IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, 2022, pp. 218–225.

[11] D. B. Bose, A. Rahman, and M. S. I. Shamim, "'under-reported' security defects in kubernetes manifests," in *2021 IEEE/ACM 2nd International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, 2021, pp. 9–12.

[12] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.

[13] T. J. Olorunlana, "Preventing amazon s3 cloud storage misconfiguration using infrastructure-as-code: A policy-enforced security framework," *International Journal of Networked and Distributed Computing*, 2026.

[14] A. Hogan, E. Blomqvist, M. Cochez, C. D'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, 2021.

[15] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proceedings of the 1998 Workshop on New Security Paradigms*, 1998.

[16] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 2002.

[17] S. Jha, O. Sheyner, and J. M. Wing, "Two formal analyses of attack graphs," in *Proceedings of the 15th IEEE Computer Security Foundations Workshop*, 2002.

[18] A. Sabur, A. Chowdhary, D. Huang, and A. Alshamrani, "Toward scalable graph-based security analysis for cloud networks," *Computer Networks*, vol. 206, p. 108795, 2022.

[19] M. Elmiger, M. Lemoudden, N. Pitropakis, and W. J. Buchanan, "Start thinking in graphs: Using graphs to address critical attack paths in a microsoft cloud tenant," *International Journal of Information Security*, vol. 23, no. 1, pp. 467–485, 2024.

[20] K. Kurniawan, A. Ekelhart, E. Kiesling, G. Quirchmayr, and A. M. Tjoa, "KRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data," *Computers & Security*, vol. 121, p. 102828, 2022.

[21] A. B. Nassif, M. Abu Talib, Q. Nasir, H. Albadani, and F. M. Dakalbab, "Machine learning for cloud security: A systematic review," *IEEE Access*, vol. 9, pp. 20 717–20 735, 2021.

[22] G. Rjoub, J. Bentahar, O. Abdel Wahab, R. Mizouni, A. Song, R. Cohen, H. Otrok, and A. Mourad, "A survey on explainable artificial intelligence for cybersecurity," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 5115–5140, 2023.

[23] F. Charmet, H. C. Tanuwidjaja, S. Ayoubi *et al.*, "Explainable artificial intelligence for cybersecurity: A literature survey," *Annals of Telecommunications*, vol. 77, pp. 789–812, 2022.

[24] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proceedings of NAACL-HLT Demonstrations*, 2016, pp. 97–101.

[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[27] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[28] OpenAI, "Gpt-4 technical report," 2023.

[29] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," 2024.